

Polyphonic Music Score Following

Sheridan Harding Christian Howell
Momoka Matsushita Jordan Sheppard

April 4, 2023

Abstract

Music score following for polyphonic music is a challenging problem because the different notes being played simultaneously—a chord—leads to harmonic interdependence between the frequencies of the notes. This means that we cannot use a form of frequency-domain decomposition of each frame or note and match it to the frequency spectrum of the observed note for some template for that instrument. Since musicians also do not always play exactly according to the score, this means the system needs to be capable of tracking the score simultaneously given the latest chord that was played with this added layer of variation. Similar to the problem computer vision seeks to solve, we wish to solve this problem of computer listening given the current snapshot of the audio signal, the ground-truth score of what should be happening at some point during the performance, and knowledge of the sequence of musical states which came before it. We investigated this problem using Hidden Markov Models, Kalman filters, and Viterbi algorithm.

1 Problem Statement and Motivation

As machine learning models increase in power, there has been a lot of time and energy spent trying to improve music score-following, i.e. having a computer follow along with sheet music as a performer plays a piece live. Polyphonic music is more difficult to follow than monophonic or homophonic music because the frequency spectra interfere with each other in both constructive and destructive ways. The problem of polyphonic musical score-following is an important open problem in music data retrieval and its uses range from sound synthesis and creating virtual scores to assisting performers with turning pages to de-soloing, or removing a solo from a recording with other instruments. [1] [5].

There exist tools that can accept audio and output the notes that were played in the piece and we have tools that can approximately follow along with music that is being played (think of piano or guitar teaching mobile apps). However, most of the tools that can extract the notes played, as well as the tools we have that work in real-time are limited to simple applications and do not predict the upcoming chords, while taking into account the artistic liberties taken on by the performer. For example, when playing trills and mordents, artists often do not play a note a prescribed number of times, choosing instead to play it as feels and sounds good to them. This makes tracking where a performer is (or should be) on a music score real-time a very difficult task.

We draw inspiration from [3] to solve this problem of music score-following for polyphonic music using Hidden Markov Models (HMM), switching Kalman filters, and Viterbi algorithm. Since our observed and hidden states are continuous but our algorithms we wished to use require discrete inputs we used a hybrid of the first two methods and we use the Viterbi as our decoder to know when to move the cursor following the score.

2 Data

We initially were planning on using audio and MIDI files from the Bach10 dataset, a collection of ten pieces by Bach. However, we decided that attempting to follow the scores for the music with many

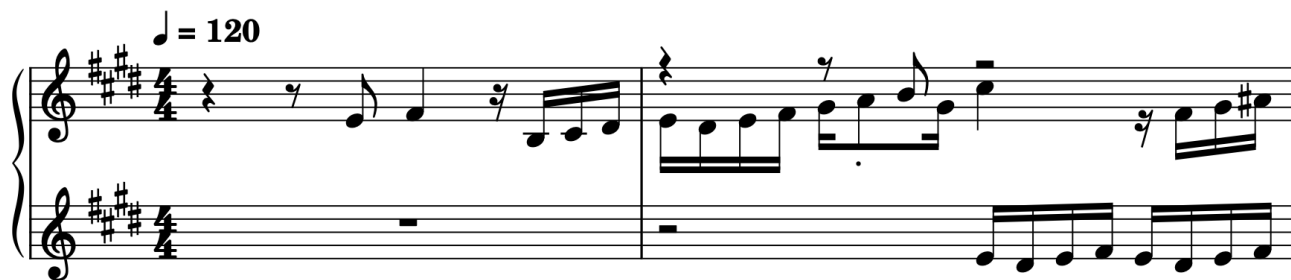


Figure 1: The first two measures of Fugue 9 from Bach’s *Well-Tempered Clavier, Book 1*. Note that the first measure contains 5 single-note chord events (excluding the rests). The second measure contains 6 more single-note chord events before a counter-melody enters in the left hand, creating a sequence of 2-note chord events for the remainder of the measure

different instruments playing simultaneously would likely prove more difficult than we could handle for a class project. To simplify the problem, we chose to work with a single instrument, namely the piano which is “one of the most challenging score following cases, due to the high degree of polyphony in the piano” [3]. Unfortunately, this meant the Bach10 dataset would no longer be the ideal source for data for this project.

We weren’t finished with Bach, though. Instead of using the Bach10 dataset, we found MIDI files for several of his preludes and fugues made available for public use at 8notes.com, as well as 48 corresponding audio files publicly available on Wikipedia, each a few minutes apiece. The data was clean, but required some preprocessing in which we parsed the MIDI files and converted them into sequences of note values, extracted information about the notes, including the onset and offset time for each, and retrieved the chords from our sequence of notes.

We did not use all of the audio data that we found on Wikipedia, but we were able to choose a sufficient number of pieces to use for training data and leave others untouched to use for testing purposes without contaminating our results.

3 Methods

There were a variety of methods which we attempted to use to solve this problem, all inspired by the literature from the earlier Motivation section.

Before embarking on a description of these methods, however, we first define what we mean by a “chord”. For the purposes of our models, a “chord” is a group of notes that are sounding simultaneously, according to what is written by a composer in the musical score. This is a bit different than the traditional, musical definition of a chord or a chord progression.

To demonstrate this, consider the example in Figure 1. The first measure contains a sequence of single-note chord events, with the first note played corresponding to the chord event [E4]. However, starting in the second measure, multiple notes are being played at a time. Each region with a different combination of notes being played at the same time is considered a unique chord event, with the chord on the beginning of the third beat of measure 2 being considered as [E4, C#5], for example.

With this definition in mind, we now turn to describing the various iterations of our approach to this problem. For this problem, we have the following elements:

- An observed sound signal—in the live score following case, this would be a live performance, but we limit our scope to recordings instead.
- A hidden “score”—the chords, rhythms, and other musical elements that the model will need to estimate.

This combination of an observed time-series and a sequence of hidden states seems to be ideal for the application of a Hidden Markov Model (HMM). Naively using an HMM is infeasible, as the observed signal is continuous, however, applying a Kalman filter is also difficult because the hidden score is actually a discrete sequence. The switching Kalman filter (SKF) is a method developed to handle such hybrid models[4]. An SKF works by running a Kalman filter on the continuous space with parameters determined by the current discrete state. That is, the filter has system and observation equations

$$\begin{aligned}\vec{x}_t &= A_t \vec{x}_{t-1} + \vec{v}_t, \\ \vec{y}_t &= B_t \vec{x}_t + \vec{w}_t,\end{aligned}$$

where $\vec{y}_t \sim \mathcal{N}(0, Q_t)$ and $\vec{w}_t \sim \mathcal{N}(0, R_t)$ are independent sources of Gaussian noise and A_t, B_t, Q_t, R_t are dependant on the current state S_t . The sequence S_t is in turn a first-order Markov Chain with a transition matrix Z and initial distribution π that determines the initial state S_1 , with $\vec{x}_1$ normally distributed with mean and covariance matrix determined by S_1 . This setup means that the initial distribution $p(X_1)$ is a mixture of Gaussians, one for each possible state. Suppose we have M possible states. Then the second distribution $p(X_2)$ becomes a mixture of M mixtures of M Gaussians, for a total of M^2 Gaussians and an exponential increase in complexity as time goes on. This can be mitigated by collapsing these mixtures into a single Gaussian at each step.

The SKF seemed a promising solution to our dilemma, but in practice it was difficult to align with the model. That, combined with the fact that the only Python implementation of the SKF that we could find was in a GitHub repository that hadn't been maintained since 2017, lead us to seek alternative methods. Specifically, we focus now on a pure HMM-based approach.

To fit this new model, we use an audio recording of a piece we would like to follow with our algorithm. We cut the audio signal up into a sequence of N frames, and take the Constant-Q transform (CQT) of each frame. The CQT is similar to the Fourier Transform, but has a logarithmic scale rather than a linear scale in the frequency domain. This allows for a higher resolution at lower frequencies, and a lower resolution at higher frequencies, which is comparable to how harmonic frequencies work in music [1]. We denote the observable sequence of frequency-bin representations of the audio at each frame i as $(Y_i)_{i=1}^N$.

Of course, the performer was playing a written piece of music during the recording, so the audio in each frame must come from one of the chords of a piece that we could find in a MIDI dataset. So, the "hidden" state $(X_i)_{i=1}^N$ at each time i represents which chord of the piece is being played in that audio frame.

There is some nuance to how the hidden state sequence is constructed. Traditionally, when HMMs are used, there is a transition matrix between different hidden states which indicates the probability of transitioning from one state to another. However, in our case, we know the sequence of chords in a piece before the performer starts playing, so in that way, the hidden state sequence (in terms of which chord follows which other chord) is already known.

However, because each hidden state X_i represents a chord being played *at the i 'th frame of audio*, one chord-state will actually last a number of audio frames before transitioning onto the next chord. To deal with this technicality, we model each chord X_i with a sequence of M_i chord evolution states $(X_{i,j})_{j=1}^{M_i}$. We describe these chord evolution states below, but we refer the reader to Figure 2 to see this process visually.

The first two chord evolution states are always the same: 2 states, with probability $p = 1$ of transitioning directly to the following state, representing the attack (or first sounding) of the chord in the recording.

The next $M_i - 2$ chord evolution states represent the sustain of the chord, or the time period where the chord is sounding. How long a note is sustained in a performance is well-modeled by a negative binomial distribution, as found in [5]. Let $n = M_i - 2$ be the number of states in the sustain phase, and p be the probability of transitioning from one sustain state to the next sequential one. Then $1 - p$ is the probability of a sustain state transitioning back to itself.

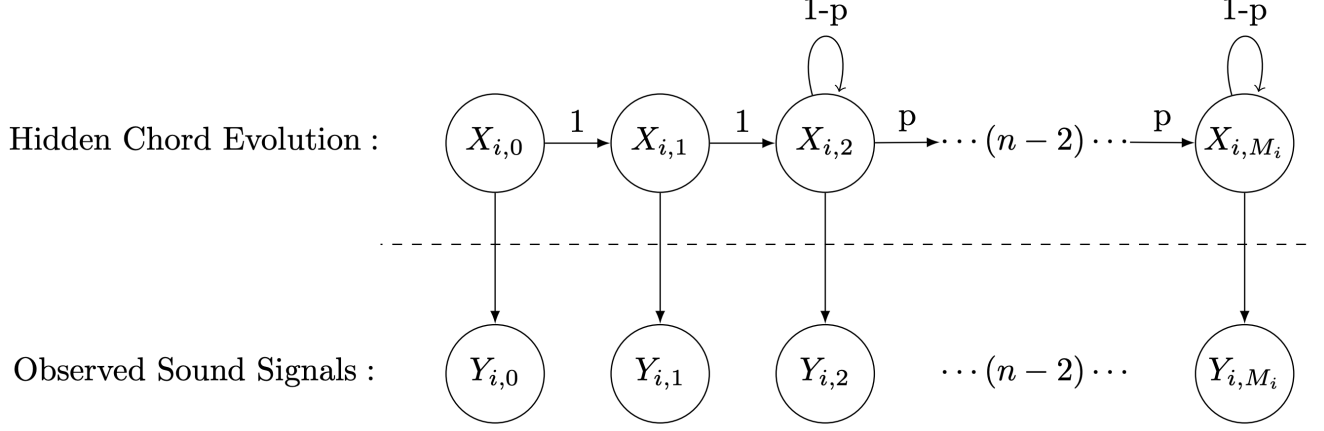


Figure 2: The evolution of one chord X_i through the attack states ($X_{i,0}$ and $X_{i,1}$) and the sustain states ($X_{i,2}$ through X_{i,M_i})

Let L_i be a discrete random variable representing the number of audio frames a chord spends in the sustain state. Then the probability of a chord sustain state lasting l frames can be given by

$$P(L_i = l) = \binom{l-1}{n-1} p^n (1-p)^{l-n}, \quad l = n, n+1, \dots$$

which has a mean $\mathbb{E}(L_i) = \frac{n}{p}$ and variance $\text{Var}(L_i) = \frac{n(1-p)}{p^2}$.

Using these tools, we use the following process to take an audio recording, with its associated score, and format it for use in our model and analysis:

- 1) Parse the audio file into a series of N frames, and transform each frame into the frequency domain using the CQT. Call this sequence $(Y_i)_{i=1}^N$.
- 2) Take the corresponding score/MIDI file, and parse it into a sequence of K chords (collections of simultaneously-sounding notes) $(X_i)_{i=1}^K$, along with their relative durations in audio frames (compared to the total number of audio frames).
- 3) For each chord $i = 1, 2, \dots, K$, do the following:
 - (i) Using the Method of Moments on the mean and variance formulas derived above, find n (the number of sustain states) and p (the probability of transitioning from one sustain state to the next) for this chord. (We fix the variance to be a relatively high value to represent our uncertainty about a given performance.)
 - (ii) Create the corresponding chord evolution state sequence $(X_{i,j})_{j=1}^{M_i}$, where $M_i = n + 2$, $X_{i,0}$ and $X_{i,1}$ are attack states with a probability 1 of transitioning from itself to the next state, and $X_{i,2}$ through $X_{i,n+2}$ are the n sustain states, each with a probability p of transitioning to the next state, and $1 - p$ of transitioning back to itself.
- 4) Construct the full hidden-state sequence $(\hat{X}_i)_{i=1}^P = ((X_{i,j})_{j=1}^{M_i})_{i=1}^K$, where $P = \sum_{i=1}^K M_i$, by appending the K separate chord evolution state sequences back-to-back.

Denoting $t_c = (i : \hat{X}_i = X_{c_0})$ to be the timestep of the first attack state for each chord $c = 1, 2, \dots, K$, we can now formally state the goal of each iteration of our model: to find t_c for every chord in the piece. Upon achieving this, we would be able to know the time to advance a cursor in the score to the next chord event.

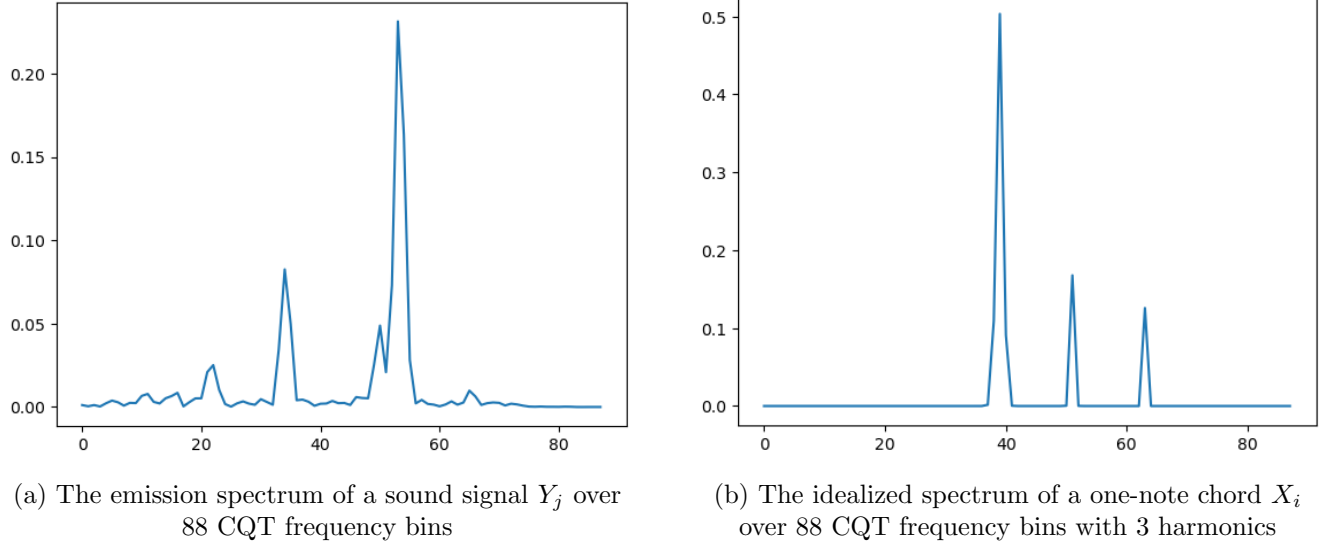


Figure 3: Comparison of a sound emission spectrum Y_j to a given possible chord X_i

Our method to attempt this problem was to use the Viterbi algorithm to get the most likely state sequence $(\hat{X}_i)_{i=1}^N$ for a given performance. This seemed like a logical option, as Viterbi tries to decode the most likely, globally-optimal sequence that a given set of data produced.

In order to compare the emission probabilities of a given chord X_i producing a given audio frame Y_j , we settled upon modeling a chord X_i as a mixture of low-variance Gaussians, with peaks at the fundamental frequency and first few harmonics created by each of the notes in X_i (see Figure 3).

We had the idea to try to create a pseudo-probability measure of the similarity between two spectra. To do this, we normalized each spectrum X_i to sum to 1. We then normalized the emissions Y_j to sum to 1 as well, and took the L2-error between each X_i and Y_j . Then, we used $E_{i,j} = e^{-\|X_i - Y_j\|}$ as our pseudo-probability (since chords with large error would get a lower $E_{i,j}$ score, and chords with small error would get a higher $E_{i,j}$ score).

Then, we used the Viterbi algorithm in the usual way, replacing $E_{i,j}$ for the emission probabilities $b_{i,j}$ that would normally be there.

4 Analysis

Our modified Viterbi method produced some interesting results. The first thing we noted was that it took an extremely long time to run the algorithm (over 90 minutes for a 3.5-minute piece). Each frame of audio took between 0.5 to 1 seconds to decode, and there were over 10,000 frames in one audio example that were originally only a few milliseconds long in the first place. This result was undesirable, as our original intent was to create an algorithm that could be used during real-time performance. Such a result suggests that our state space might be too large to use a full scale Viterbi-style method, as there were too many computations to make for each audio frame to make it run in any reasonable amount of time.

The results that the algorithm spat out were also very interesting, and very incorrect. The model correctly followed the silence before the piece started, and correctly followed the first few notes of the piece. After that, however, the follower got lost, and zoomed as fast as it could to the end of the piece. So the initial results of the algorithm seemed promising, but somewhere in the mathematics, the follower decided that the optimal sequence was nowhere near the actual states happening at a given point in time.

While it is hard to quantify the results we got with numbers (since the dataset for piano did not have ground-truth time-alignments from a performance to each chord in its corresponding score that we could compare our alignments to), qualitatively, this is extremely bad performance.

There are a few possible things that were incorrect or oversimplified in our modeling process, which might have caused these errors.

First of all, we assumed that a performer was actually performing each piece perfectly, with no major deviation or variation. This is an inherently flawed assumption, as there are many times that performers will take artistic liberties with a piece (such as trying to linger longer than written on certain chords for artistic effect, or adding a few extra reverberations to a trill to add flair to the performance). These liberties were not accounted for in our very simple model for exact note-level following with a fixed-variance approach to the length of each sustain phase (even if that variance was very high, it still does not produce a great model).

In addition, our chord-evolution model was not very well-designed for short, quick notes. Longer notes are sustained for a while, so the self-loops (and requiring that a note hit every intermediate state) were justified. However, if a 32nd-note were played at a fast tempo, for example, it would only be sounding for a handful of milliseconds at most. So, it would only be present for a very small number of audio frames in our model. As a result, modeling with a way to get to the end state of a short chord from every other intermediate sustain state of the chord might be more appropriate for these shorter, more transient chord structures.

Another source of error in our modeling process could have come from how we modeled each chord as a mixture of Gaussians, and compared the frequency spectra of each sound frame to the normalized mixture-model as a proxy for emission probabilities using the L2-error. This is inherently a very flawed idea—the idea of using error as a probability measure violates many of the assumptions about a probability space (such as independence of non-correlated events under our probability function, for example). As a result, inserting the normalized error measure into a Viterbi algorithm in place of actual probabilities significantly over-estimated the chance that a certain chord produced a totally unrelated sound spectrum (all due to the fact that the L2 error was not that bad overall—just in a few places that really mattered). This would have significantly increased the chances that the score follower would just rush to the end state, since there was no way to limit how truly probable an emission from an unrelated chord was using L2-error.

Finally, another possible modeling error that could have led to the overall computational complexity of our model was the size of our state space. In reality, if we considered ourselves to be most probably in a certain state/playing a certain chord at a time t_i , then we should only really be considering the emission spectra from a window of states nearby that current state as allowable emissions. If we were to alter the Viterbi algorithm to be more localized in this manner, it would be more suited to analyze music (which has a very specific local structure, and a harder-to-infer global structure). This might reduce both the number of computations needed at each timestep, as well as reduce the likelihood of a non-related chord being predicted as the most likely to have produced a given emission, leading to the rushing of the algorithm through the score.

All of these considerations were fundamentally flawed or ignored by our model, which led to a lot of error in the algorithm’s output. It would take a lot of time and more research to be able to overcome these considerations (like possibly finding a better proxy for emission probability, using more developed and advanced techniques than we discussed in this curriculum), but it is very possible to alter our model to become more suitable to this problem.

5 Ethical Considerations

While this project may not have some of the major ethical concerns that could be found in, perhaps, facial recognition or image detection software, there are certain other considerations that we must take into account.

When dealing with public performances, we must always keep in mind that transcription of copyrighted music without permission is illegal. Were this software to be used simply for detecting when an automatic

page-turner should turn a page, this may not be a problem, but if it were used by a member of the audience, there could be some ethical concerns.

Fully-functioning and accurate software for score-following could also potentially be used for evaluating or scoring performances or competitions. On the one hand, this could make the judging process more objective, as we can see exactly when wrong chords were played, notes were held for too long, or the performance was rushed. This does not take into account, however, the performers' intentional and artistic renderings of the piece, inserted to express emotion and beauty. In order to guarantee a fair score at an evaluation with implementation of this software, performers must be aware ahead of time how they could be affected by using these techniques being used as a judging method. This could also lead performers to strive to be more exact in their timing during performances rather than playing it in a way where they are comfortable and can showcase their artistic expression.

Furthermore, any time we are recording another person or collecting data, privacy is always a concern. Thus, when using score-following software, it is essential to get the consent of the (fully-informed) participant before any data is collected. Upon collecting that data, it is important to use it only for the purposes of which the participant was informed prior to their consent and it must be kept safe from other parties that may wish to use it for any other motives.

6 Conclusion

Piano score-following is a difficult task into which much research has been performed and much more is likely needed. We began by hoping to be able to follow the score for a piece with multiple instruments playing at the same time. We then simplified this to be a single piano, realizing that the polyphony of the piano is complicated enough by itself. We then developed many different algorithms attempting to accurately predict when a note was about to change. While our project did not perform as well as we would have liked, we cannot be too disappointed with our results considering the constraints under which we were working, and there were some hints we saw that seem to imply we were on the right track (or at least not too far off of a right path).

We were not able to follow along with entire pieces of music, but there does seem to be some potential that our algorithms, with relatively minor (or slightly more major) changes, may end up being successful given simple enough music. So, although there is a chance that our algorithms are incorrect and have no true capacity for predicting when to switch notes, it actually seems more likely that they have some basis behind them and, rather than requiring a complete upheaval of our computations, it may be enough to simply adjust our existing calculations, as well as adding some new ones as we see fit to increase accuracy.

Supposing we were able to get our algorithm working properly for following piano music, it would likely still be quite difficult and require a lot of training time in order to extend this to other polyphonic instruments and especially to multi-instrument ensembles. This is likely to be an open area of research for many more years, but with increasingly powerful technology and predictive algorithms, we do believe that a solution is not beyond the realm of possibility.

References

- [1] Lin Hao Lee. Musical score following and audio alignment. *arXiv e-prints*, pages arXiv–2205, 2022.
- [2] Yucong Jiang. *Piano Score Following with Hidden Timbre or Tempo Using Switching Kalman Filters*. Indiana University, 2020.
- [3] Yucong Jiang and Christopher Raphael. Piano score-following by tracking note evolution. *Cited on*, page 3, 2019.
- [4] Kevin P Murphy. Switching kalman filters. 1998.

- [5] Christopher Raphael. Music plus one and machine learning. In *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, pages 21–28, 2010.